

# Final Project

## CSE 5539, AU2026

### Important Dates

- **Proposal Due:** September 28, 11:59pm
- **Check-ins:** TBD
- **Presentations:** TBD
- **Final Report Due:** TBD

## 1 Collaboration

You must work in teams of 2 or 3. Smaller teams can be less ambitious in scope but should not be less complete: a half-implemented system does not make a good project outcome. All partners are expected to contribute equally to the submission. You are required to detail all your contributions in your reports which will determine your grade. You may collaborate with a person from outside the course as well in case you're also using this final project for another course. You are also free to discuss your project with others in the course, though only the people on your team should contribute to the actual implementation/experimentation involved. Any external resources used must be clearly cited.

## 2 Combining with Other Projects

You may combine this project with other research or coursework projects, provided that it is aligned with theme of this course. Non-language data is allowed if it has language-like structures or it is combined with language like data (e.g., genomics, time-series data).

## 3 Overview

This project is an independently conducted original study of some problem in language modeling as it relates to the theme of the course, which is “constrained driven progress in language modeling research”. The theme is pretty broadly interpreted; if you are not clear if your project idea is related to the theme, shoot me a message to ask. You might do a more engineering-style project: pick a task and a dataset, design or expand on some model, and try to get good results. You can also do a more analytical project: pick some problem and try to characterize it in greater depth. What does the data tell us? What does this tell us about language or about how we should design our NLP systems? What can interpretation techniques, contrast sets, or other focused evaluation measures tell us about how models are doing? Most contributions appropriate for an NLP/ML conference or workshop paper are in scope here as long they relate to the theme. Your end goal shouldn't be just reimplementing what others have done. However, implementing someone else's model or downloading and running an existing model are great first steps and might end up getting

you most of the way there. Implementing a couple of approaches in order to gain some insight from comparing them can be a good project (somewhere between research and reproduction, and that's fine!). For projects in this area, you should start with literature search and include that in your project proposal to make sure you're not missing relevant prior work. This project is not graded on how good your results are, as long as you can convincingly show that you've done something: trained an effective model, collected some quality data, etc. Start with baby steps rather than implementing your full approach from scratch: build baselines and improve them in a direction that will eventually take you towards your full approach. You should think these steps through in your proposal.

## 4 Deliverables

**Proposal (10 points)** You should turn in a 1-2 page proposal (using a LaTeX Template: <https://github.com/acl-org/acl-style-files>) on the proposal due date. This proposal should outline what problem you want to address, what dataset(s) you plan to use, prior work, and a rough plan for how you will pursue the project. While you don't need a full related work section, you should mention the most relevant prior work you've found and state how your project relates to it. I will then provide feedback and guidance on the direction to maximize the project's change of succeeding.

**Structure** Your proposal should aim to answer the following questions (You are allowed to deviate from this structure if you like):

1. Describe the main goal(s) of your project. If possible, try to phrase this in terms of a scientific question you are trying to answer – e.g., your goal may be to investigate whether a particular model or technique performs well at a certain task, or whether you can improve a particular model by adding some new variant, or (for theoretical/analytical projects), you might have some particular hypothesis that you seek to confirm or disprove. Otherwise, your goal may be simply to successfully implement a complex neural model, and show that it performs well on a given task. Briefly motivate why you chose this goal – how does it relate to the theme of the seminar, why do you think it is important, interesting, challenging and/or likely to succeed? If you have any secondary or stretch goals (i.e. things you will do if you have time), please also describe them.
2. What NLP task(s) will you address? Describe the task clearly (i.e. give an example of an input and an output, if applicable).
3. What data will you use? Specify the dataset(s) you will use (including its size), and describe any preprocessing you plan to do. If you plan to collect your own data, describe how you will do that and how long you expect it to take.
4. What neural method(s) are you planning to use? Describe the models and/or techniques you plan to use. If you plan to explore a variant to a published method, focus on describing how your method will be different. Make it clear which parts you plan to implement yourself, and which parts you will download from elsewhere. If there is any part of your planned method that is original, make it clear.

5. What baseline(s) will you use? Describe what methods you will use as baselines. Make it clear if these will be implemented by you, downloaded from elsewhere, or if you will just compare with previously published scores.
6. How will you evaluate your results? Specify at least one well-defined, numerical, automatic evaluation metric you will use for quantitative evaluation, if applicable. What existing scores will you be comparing against for this metric? For example, if you're reimplementing or extending a method, state what score(s) the original method achieved; if you're applying an existing method to a new task, mention the state-of-the-art performance on the new task, and say something about how you expect your method to perform compared to other approaches. If you have any particular ideas about the qualitative evaluation you will do, you can describe that too.
7. What are the ethical challenges and possible societal risks of your project, and what are mitigation strategies? Try to be a little specific and intentional in describing ethical concerns (e.g. more depth than 'there will be issues with user data privacy'), and especially around issues specific to your project. Writing clear limitations and identifying potential risks is something that we're encouraging as part of a broader trend in the research community to include such statements as part of published research.

**Grading:** 10 points for turning in a proposal meeting a minimum level of coherence and quality. You are not evaluated on how good the idea is—this is a stage to get feedback and refine things. Using AI systems to help ideate is fine but please DO NOT submit AI slop / completely AI-generated proposals. It will result in a failing grade. This applies to all project related submissions.

**Check-in (15 points)** You should turn in a check-in on the check-in due date. This should probably be 2-4 pages in length and look like a fleshed out version of the proposal, building towards your final project. You should respond to feedback from the proposal stage, include a more detailed discussion of related work, and most critically, have some kind of preliminary results for your approach. Think about what you can do to make sure you have something to show at this stage, even if it's more unit test-style validation that your code is correct.

**Grading:** You will lose points on the check-in if we perceive that you have not dedicated time to flesh out the project beyond the proposal and take a first stab at the work on it.

**Final Report (65 points)** The primary deliverable is a paper written in the style of an ACL, NeurIPS, etc. conference submission. It should begin with an abstract and introduction, clearly describe the proposed idea, present technical details, give results, compare to baselines and discussion of the results, and cite sources throughout (you'll probably want to cite at least 5-10 papers depending on how broad your topic is).

The paper should be the order of 8 pages excluding references; If you have lots of analysis and discussion yielding full-page figures or are trying something more ambitious, your paper might be longer; if you're implementing something complex but succinctly described, your paper might be shorter. Critically, you should approach the work in such a way that success isn't all-or-nothing. You should be able to show results, describe some successes, and analyze why things worked or didn't work beyond "my code errored out." Think about structuring your proposal in a few phases (like the projects) so that even if everything you set out to do isn't successful, you've at least gotten something working, run some experiments, and gotten some kind of results to report.

**Grading:** We grade projects starting from a “base score” of 55/65 for projects that are executed according to the specification laid out here. This score will then be moved up or down based on the following criteria:

- **Scope:** Was the scope appropriate for the project? You will gain or lose points depending on our subjective assessment of the overall work done and whether it fits the charge you were given.
- **Clarity/Writing:** Your paper should clearly convey a core idea/hypothesis, describe how you tested it/what you built, and situate it with respect to related work. Doing a thorough job of this will raise your grade. Your grade will be lowered if you fail to explain what you did in a clear fashion or do not present results in a coherent way.
- **Implementation/Soundness:** Is the idea technically sound? Do you describe what seems like a convincing implementation? Is the experimental design correct?
- **Results/Analysis:** Whether the results are positive or negative, try to motivate them by providing examples and analysis. If things worked, what error classes are reduced? If things didn’t work, why might that be? What aspects of the data/model might not be right? If you’re writing a paper that revolves around building a system, you should try to report results for a baseline from the literature, your own baseline, your best model, and possibly results of ablation experiments.

#### 4.1 Final Presentation (10 points)

During the last week of class, every group will give a 10-15 minute presentation on their project (depending on the number of groups). This presentation should state the problem, describe the methodology used, and give highlights of the results. Because the project reports won’t have been due yet, these results might be preliminary, but should be nonzero. You will want to keep this presentation focused on high-level aspects of your approach, methodology, and results/examples of the output.

**Grading:** The final presentation should be clear and fit within the allotted time limit. It should describe your methodology, related prior work, and preliminary results or analysis.

#### 4.2 Choosing a Topic

The following is a (non-exhaustive!) list of possible directions for projects, just a few to give you some pointers. Projects may investigate fundamental questions about language models, develop or improve methods, or apply models to a particular problem. Application-focused projects are welcome: you should identify what makes your setting challenging, motivate your design choices, and evaluate how well your approach addresses those challenges.

- **Efficient Reasoning and Test-Time Computation**

Language models can spend additional computation at inference time by generating longer reasoning traces, sampling multiple answers, or revising their outputs. This project investigates how to use that computation effectively. Possible directions include comparing reasoning strategies under a fixed budget, adapting the amount of computation to problem difficulty,

or developing criteria for when a model should stop reasoning. You can evaluate trade-offs between accuracy, response length, latency, and computational cost across different tasks.

- **Learning from Rewards and Feedback**

Improving language models through reinforcement learning or supervised fine-tuning depends on the feedback available during training. This project explores how the quality, quantity, or form of feedback affects model behavior. Possible directions include studying imperfect reward functions, comparing outcome-based and intermediate feedback, or investigating how to select useful training examples. You can conduct experiments on small models and verifiable tasks, evaluating learning efficiency, generalization, and whether models exploit weaknesses in the feedback.

- **Evaluating and Improving Long-Context Modeling**

Supporting a long context window does not necessarily mean that a model can use all the information within it effectively. This project investigates how models retrieve, combine, and reason over information in long inputs. Possible approaches include retrieval-augmented generation, context compression, document reordering, and hierarchical summarization. You can examine how performance changes with input length, distracting information, or the location of relevant evidence, and investigate methods that improve accuracy while reducing the amount of context processed.

- **Memory and Long-Horizon Tasks**

Tasks involving many steps require models to preserve useful information, track progress, and remain consistent with earlier decisions or instructions. This project explores memory and planning strategies for extended interactions or long-form generation. Possible approaches include maintaining structured notes, summarizing previous interactions, retrieving relevant history, or decomposing tasks into smaller stages. You can evaluate these methods on tasks such as document writing, conversational assistance, or interactive problem solving, analyzing how errors accumulate as tasks become longer.

- **Alternative Architectures for Language Modeling**

Different model architectures make different trade-offs between computation, memory, and the information they can retain. This project investigates alternatives to standard Transformer architectures, such as sparse attention, recurrent models, state-space models, or hybrid approaches. You can implement small models or extend existing implementations to study particular capabilities, such as retrieving distant information or generalizing to longer sequences. Evaluation should consider both task performance and resource requirements, with controlled comparisons that help explain architectural strengths and limitations.

- **Model Compression and Efficient Inference**

Deploying language models with limited hardware or strict latency requirements often requires reducing their computational and memory costs. This project explores techniques such as quantization, distillation, pruning, speculative decoding, or input compression. You can investigate when these techniques preserve performance and which tasks or inputs are most affected. Possible extensions include adapting the amount of compression to the input or combining complementary methods. Evaluation can include accuracy, latency, memory usage, and analysis of changes in model behavior.

- **Reliable Tool Use and Language Model Agents**

Language models can use external tools to access information, perform calculations, or take actions, but successful tool use requires deciding when and how to use them. This project investigates tool selection, planning, or recovery from tool failures. You can develop an agent for a particular task or study its behavior in a controlled environment with unavailable tools, ambiguous descriptions, or incorrect results. Evaluation can consider task completion, the number and cost of tool calls, and robustness to unexpected situations.

- **Evaluation with Limited Human Supervision**

Evaluating open-ended model outputs can require substantial human effort, while automated metrics and model-based judges may miss important errors. This project explores how to evaluate language models when reliable human labels are limited. Possible directions include studying biases in automated judges, developing task-specific evaluation criteria, or combining automated evaluation with targeted human review. You can compare evaluation methods against a small human-annotated dataset and analyze when they disagree, including their sensitivity to style, verbosity, factual errors, or answer ordering.

- **Scientific and Domain-Specific Assistants**

Applying language models to specialized domains requires handling unfamiliar terminology, finding relevant evidence, and producing answers that can be checked against reliable sources. This project explores applications such as scientific literature synthesis, extracting information from research papers, answering questions over technical documents, or supporting a domain-specific workflow. Possible approaches include retrieval, structured generation, and adaptation with limited domain data. You can evaluate factual accuracy, evidence coverage, citation quality, and the system's ability to recognize when the available information is insufficient.

- **AI Safety and Alignment**

Language models may behave in unintended ways when faced with conflicting instructions, adversarial inputs, or unfamiliar situations. This project explores methods for evaluating and improving their safety and alignment with user intentions. Possible directions include studying robustness to prompt injection, distinguishing trusted instructions from untrusted content, investigating how fine-tuning affects safety behaviors, or developing safeguards for tool-using agents. You can evaluate both the effectiveness of these safeguards and their impact on legitimate tasks, analyzing trade-offs between preventing harmful behavior, avoiding unnecessary refusals, and preserving model usefulness.

- **Code Understanding and Generation**

Coding applications require models to satisfy precise specifications and often provide opportunities to evaluate outputs through execution. This project explores tasks such as code generation, debugging, program repair, test generation, or understanding an unfamiliar repository. You can investigate how models use execution feedback, retrieve relevant code, or revise solutions after failed tests. Evaluation can consider functional correctness, generalization to unfamiliar problems, computational cost, and whether improvements hold beyond the particular tests available to the model.

- **Multimodal and Accessible Language Applications**

Applications involving documents, images, or speech require models to combine information across modalities while preserving details that matter to users. This project explores tasks such as answering questions about visually complex documents, describing images, or building speech-based interfaces. You can investigate how input quality, resolution, compression, or language affects performance, and develop approaches for a particular accessibility need or usage setting. Evaluation can include task accuracy, faithfulness to the input, robustness to noise, and computational requirements.

- **NLP for Low-Resource Languages**

Many state-of-the-art NLP models are trained primarily on high-resource languages, leaving low-resource languages underrepresented. This project investigates transfer learning, multilingual embeddings, and zero-shot/few-shot adaptation techniques for improving NLP performance on underrepresented languages. The goal of this line of work is to bridge the language gap and improve accessibility.

- **NLP for Social Good**

NLP has the potential to address real-world problems such as misinformation detection, mental health analysis, and disaster response. This project explores socially beneficial applications of NLP. Ethical considerations, such as bias mitigation and fairness, will also be an important aspect of the project. You can develop models using real-world datasets and assess their performance in socially relevant contexts.

- **Task-Specific Prompt Optimization**

Prompt engineering is crucial for improving the effectiveness of large language models on various NLP tasks. This project investigates different prompt optimization techniques. You can experiment with adversarial prompts to analyze model robustness or use meta-learning to automate prompt selection. A key outcome of this project is understanding how different prompt structures influence model behavior and performance.

- **Continual Learning for Language Models**

Unlike static training, continual learning enables models to adapt to new data while limiting catastrophic forgetting of prior knowledge. This project explores lifelong learning strategies such as replay-based memory, dynamic architectures, and modular adaptation. You can experiment with evolving datasets, such as news streams, and analyze how models retain previously learned knowledge while incorporating new information.

**CAUTION** Your project probably should not revolve around needing to fine-tune large language models on really large-scale datasets (100k+ examples). Even getting a very basic version of this working is tough without access to significant other compute resources. Needing to train a model like this once is okay, but expecting to do it 10+ times and iterate on your results will prove challenging unless you have a lot of experience with these types of systems and access to good GPU resources. Exceptions are allowed in case you do have access to your own GPU resources.

## 5 Computational Resources

This course has an allocation on OSC. Please find information on Teams on how to add yourself. Contact us if you face any issues. Google Cloud Platform or Microsoft Azure is also a way to

get GPU time, either for free when you sign up (although the nature of this promotion changes sometimes) or for relatively inexpensive. Colab Pro (for \$9.99/mo) is also an option that makes GPUs available to you. Unfortunately, we are not able to provide access to LLM APIs like OpenAI or Anthropic (they might be available through Azure AI with free credits). However, many projects may be doable for only modest cost (less than \$20 in expenditures). You can also use AI assistant Web Interface, though note that it may be unreliable and results not reproducible. Do not wait until late in the project to sort access issues out, as availability of all of these services is subject to change.

## 6 Submission Guidelines

You should submit a single PDF for proposals on Canvas.

For other submissions, details TBD.